

Assignment-01 Report

[Pooool]

Programmazione Concorrente e Distribuita [25-26]

Alex Mazzoni

`alex.mazzoni3@studio.unibo.it`

May 11, 2026

Contents

1	Analisi del Problema	2
1.1	Input asincroni e reattività dell'EDT	2
1.2	Parallelizzazione delle collisioni	2
2	Design e Implementazione	3
2.1	Architettura del Sistema (MVC e Observer)	3
2.2	Gestione asincrona degli Eventi e Game Loop	3
2.3	Concorrenza delle Collisioni	3
2.3.1	Sincronizzazione tramite Barriera	4
2.3.2	Ottimizzazione Locale	4
2.3.3	Lock Ordering: prevenzione da Deadlock	4
2.4	L'approccio Task-based, con meccanismi high-level	5
3	Modellazione tramite Reti di Petri	6
3.1	ActiveController e Polling asincrono	6
3.2	Collisioni Multithreading	7
3.3	Prevenzione Deadlock tramite Lock Ordering	8
4	Benchmark e Performance	9
4.1	Analisi dei Risultati	9
4.2	Confronto con Speedup	10
5	Verifica del Progetto	11
5.1	Casi di Studio Applicati in JPF	11

1 Analisi del Problema

L'implementazione di *Poool* nasce dall'unione dei due sketch: `sketch01`, orientato alla logica sequenziale, mentre `sketch02`, orientato all'interazione asincrona in architettura MVC. Durante questa integrazione sono emerse alcune criticità:

1.1 Input asincroni e reattività dell'EDT

L'**Event Dispatch Thread (EDT)** di Swing deve restare reattivo: ogni operazione bloccante sull'EDT produce freeze percepibili della GUI. È necessario l'utilizzo di un thread separato, per garantire sia l'aggiornamento della View, sia la gestione degli input asincroni.

Il problema è risolto tramite l'architettura MVC con observer: implementando un **Controller** in cui i comandi utente devono essere trasferiti dalla View ad esso senza perdere eventi e senza introdurre attese indefinite. Nello `sketch02` viene utilizzato un **AutonomousUpdater**, un thread dedicato ad aggiornare periodicamente lo stato di un Counter. Ho trovato similarità tra questo pattern e la necessità di un loop di gioco che avanzi a cadenza regolare, indipendentemente dagli input.

1.2 Parallelizzazione delle collisioni

La fase più costosa del motore è la risoluzione delle collisioni tra palline, basata su confronti a coppie. Con n palline, il costo cresce quadraticamente come $\mathcal{O}(n^2)$. Negli scenari ad alta densità (la configurazione **massive**: 4500 palline), un calcolo sequenziale non riesce a mantenere la stessa fluidità dei casi con meno palline, rendendo necessaria una parallelizzazione mirata del solo sottoproblema delle collisioni. Un primo approccio efficace alla parallelizzazione consiste nella partizione della lista di palline, garantendo uniformità al calcolo delle collisioni tra i thread worker simultanei.

Coordinamento e Coerenza

Parallelizzare le collisioni assegnando blocchi di palline a thread worker è solo una parte della soluzione; è necessario anche coordinare i thread per garantire che il game loop resti coerente. Affinché il rendering mostri uno stato complessivo corretto e veritiero risulta fondamentale imporre un punto di ritrovo o attesa condiviso (sfruttando il concetto di **Barrier**). Questo vincolo assicura l'integrità del game loop.

Rischi di Deadlock nelle Collisioni

La risoluzione concorrente su coppie condivise di palline richiede lock multipli. Senza una politica deterministica di acquisizione, due thread potrebbero ottenere i lock in ordine inverso, causando attesa circolare e quindi deadlock.

Per prevenire questa condizione viene applicato un lock ordering globale sulle palline: i monitor sono acquisiti sempre nello stesso ordine logico. La validità della scelta è stata poi verificata formalmente con JPF 5.1.

2 Design e Implementazione

È stata progettata un'architettura modulare e scalabile, con particolare attenzione alla gestione, al riuso della logica di gioco e della View. Il package *common* contiene il dominio condiviso, in modo che le due versioni **V1** e **V2** possano differire solo per il **Controller** e l'implementazione del **CollisionResolver**, difatto in fase di bootstrap, possono essere specificati, oltre alla strategia di generazione delle palline.

2.1 Architettura del Sistema (MVC e Observer)

L'implementazione per una visualizzazione reattiva 1.1, riprende `sketch02` e si basa su **Model-View-Controller (MVC)** con pattern **Observer**.

Per garantire la thread-safety, la rappresentazione visuale passa attraverso un **View-Model**. Esso viene frequentemente aggiornato con dati differenti, rendendo così separati i dati dei model, soggetti ad update frequenti, e sincronizzazioni tra i vari thread. In questo modo la View legge una vista coerente senza accedere direttamente alle strutture mutabili interne ai modelli.

2.2 Gestione asincrona degli Eventi e Game Loop

I **Controller** sono stati progettati per aggiornare in maniera costante la logica di gioco, gestire gli input e notificare la View.

Nella versione implementata con meccanismi low-level **V1**, usa **ActiveController**, il thread utilizza un **BoundedBuffer**, di dimensione limitata, per ricevere i comandi di gioco (input del giocatore). Il controller non resta bloccato indefinitamente su una *get()*, utilizza invece *poll(waitMs)*.

Se un **GameCommand** è presente in coda, il comando viene eseguito; altrimenti il loop avanza sequenzialmente. In questo modo input e simulazione convivono nello stesso thread logico mantenendo una cadenza stabile.

La versione **V2** con **ExecutorController** mantiene un loop concettualmente simile, facendo però uso di costrutti high-level. Ad esempio, al posto di *BoundedBuffer* per i comandi, viene utilizzato **ConcurrentLinkedQueue**, usando *offer* che a differenza della *put* non è bloccante.

2.3 Concorrenza delle Collisioni

La risoluzione collisioni su N palline è il punto in cui la concorrenza produce il massimo impatto, ma anche i principali rischi di contesa. L'interfaccia *resolveCollisions* è implementata in diverse versioni, di cui concorrenti: **MultithreadingCollisions** e **ExecutorCollisions**, mentre **SequentialCollisions** è la versione non concorrente, implementata in `sketch01`.

2.3.1 Sincronizzazione tramite Barriera

L'implementazione della classe `MultithreadingCollisions` implementata nella versione V1, utilizza meccanismi di low-level. I thread vengono istanziati una sola volta all'avvio e tenuti sempre in attesa tramite un ciclo continuo. Per coordinarli in modo sicuro con il game loop principale, sono state utilizzate due barriere cicliche. La prima barriera funziona come un cancello di partenza, bloccando i worker finché il master non ha preparato e condiviso i dati del nuovo frame. Appena ricevono il via tramite la funzione `hitAndWaitAll()`, i thread si spartiscono in modo deterministico la lista delle palline elaborando le collisioni parallelamente, lavorando su porzioni separate senza ostacolarsi su strutture condivise.

Calcolate tutte le coppie di collisioni assegnate, ogni thread si ferma sulla seconda barriera. Solo quando tutti i partecipanti raggiungono questo traguardo finale, il master ha la certezza che la fisica del frame è completa e sblocca il ciclo per procedere al rendering. Questo design azzerava l'enorme peso sul sistema operativo e sul Garbage Collector, permettendo al gioco di mantenere un frame rate fluido anche nella configurazione con migliaia di palline.

La **CyclicBarrier** è stata implementata sfruttando direttamente i monitor nativi di Java tramite i metodi `synchronized`, `wait()` e `notifyAll()`. La classe tiene traccia di quanti thread sono attesi, quanti sono effettivamente arrivati e a quale ciclo di esecuzione ci troviamo. Quando un worker arriva alla barriera, incrementa il contatore. Se mancano ancora altri thread all'appello, si mette in `wait()`, all'interno di un ciclo `while`, fondamentale per evitare che il thread riparta per sbaglio a causa di un risveglio spurio. Appena l'ultimo thread completa il suo lavoro, la barriera si resetta (definiamo così una *barriera ciclica*), e chiama un `notifyAll()` per svegliare tutti i worker in attesa, facendoli ripartire.

2.3.2 Ottimizzazione Locale

La funzione `Ball.checkAndResolveCollisionSafe` esegue prima un pre-check geometrico senza lock. Questa scelta scarta il maggior numero di coppie, considerando solo quelle vicine. Riducendo così il numero di lock inutili su coppie sicuramente non in collisione.

2.3.3 Lock Ordering: prevenzione da Deadlock

Le collisioni devono essere risolte garantendo la sezione critica su entrambe le palline, e più thread potrebbero tentare lock multipli in ordine inverso. Per evitare deadlock si applica un criterio deterministico di **Lock Ordering**. Si confronta l'hash d'istanza della JVM fornito dal metodo:

```
if (System.identityHashCode(first) < System.identityHashCode(second))
```

Ogni thread acquisisce prima il lock della pallina con identificativo inferiore e poi il secondo lock, in questo modo si elimina la possibilità di attese circolari.

2.4 L'approccio Task-based, con meccanismi high-level

L'evoluzione finale adotta un modello task-based utilizzando il Framework Java Executor. Vengono create micro-task e assegnate a un pool fisso di worker, la logica di suddivisione tra le coppie rimane la stessa della versione in `MultithreadingCollisions`. Ogni task elabora parallelamente e la sincronizzazione globale è ridotta alla sola barriera finale (`invokeAll`), tornando poi al flusso della Logica principale.

Questa scelta riduce sensibilmente la contesa sui monitor e sfrutta meglio i core disponibili, i meccanismi di high-level, riducono il rischio di errori di sincronizzazione, migliorano la manutenibilità del codice, e possiamo notare anche un miglioramento nella fluidità generale del gioco 4.1.

3 Modellazione tramite Reti di Petri

Per descrivere e validare formalmente la logica di sincronizzazione e controllo, sono state sviluppate due Reti di Petri focalizzate sugli aspetti critici del sistema.

3.1 ActiveController e Polling asincrono

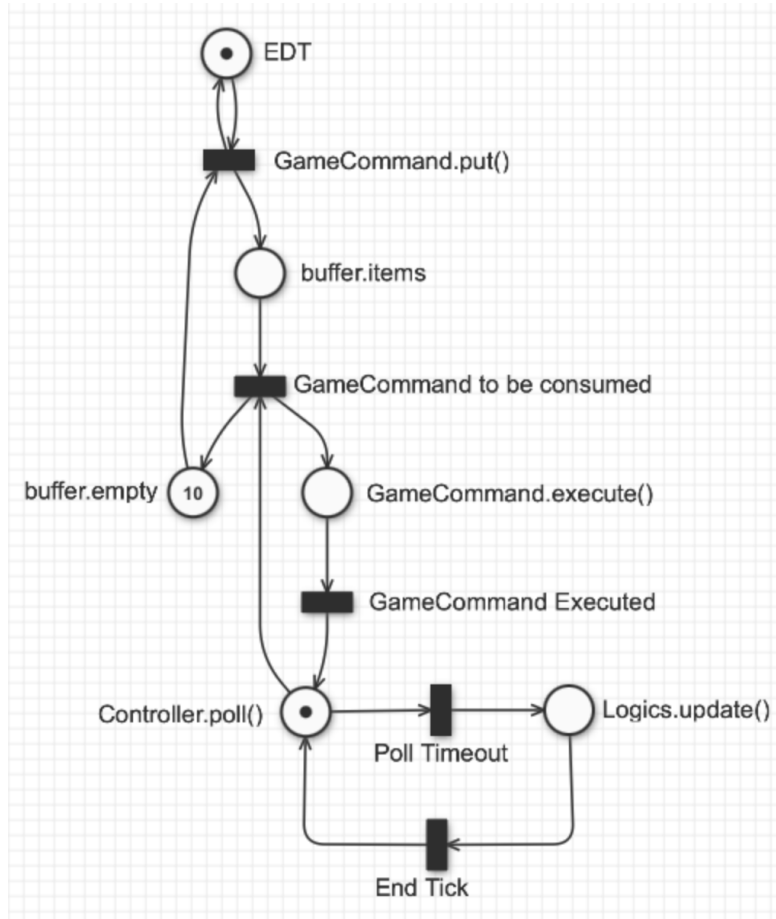


Figure 1: Rete di Petri: interazione tra EDT e ActiveController tramite BoundedBuffer.

La rete in Figura 1 astrae il coordinamento tra la UI e il `GameLoop`. L'analisi formale verifica le proprietà di **k-Boundedness**, poichè il numero di token in `buffer.items` è limitato dalla capacità assegnata (in `buffer.empty`). La **Liveness** della rete dimostra la totale assenza di deadlock. La transizione autonoma di `Poll(waitMs)` evita attese infinite in caso di buffer vuoto, garantendo che i *tick* fisici vengano ciclicamente eseguiti.

3.2 Collisioni Multithreading

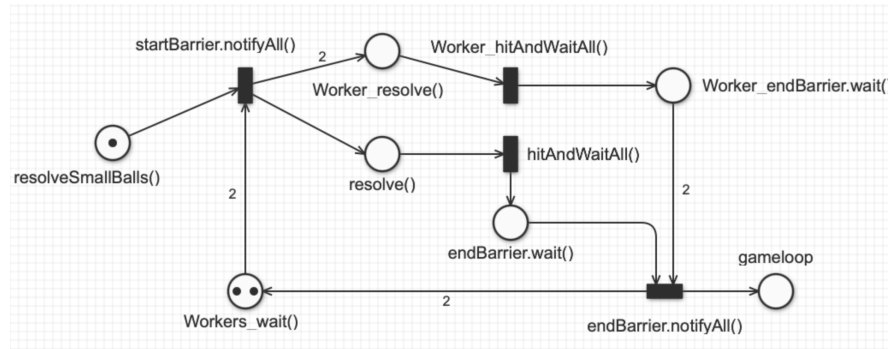


Figure 2: Rete di Petri: Risoluzioni delle collisioni con CyclicBarrier.

La Figura 2 astrae il meccanismo di risoluzione delle collisioni utilizzando un **CyclicBarrier**. La rete evidenzia la sincronizzazione tra il thread Master e i Worker durante la fase di calcolo delle collisioni. La proprietà di **Liveness** è garantita dalla struttura ciclica della rete ogni Worker e il Master, una volta completato il proprio lavoro, attende sulla barriera; La rete mostra come è necessario che tutti i token (pari numero dei worker e del Master) raggiungano la barriera prima che essa notifica il risveglio, e l'**ActiveController** possa procedere con un successivo **Gameloop**, assicurando così la coerenza dello stato del gioco prima di ogni rendering. Il numero di token e i pesi delle transizioni sono impostati a 2 ma possono essere scalati per rappresentare un numero arbitrario di Worker, mantenendo invariata la logica di sincronizzazione e garantendo sempre le stesse proprietà formali.

3.3 Prevenzione Deadlock tramite Lock Ordering

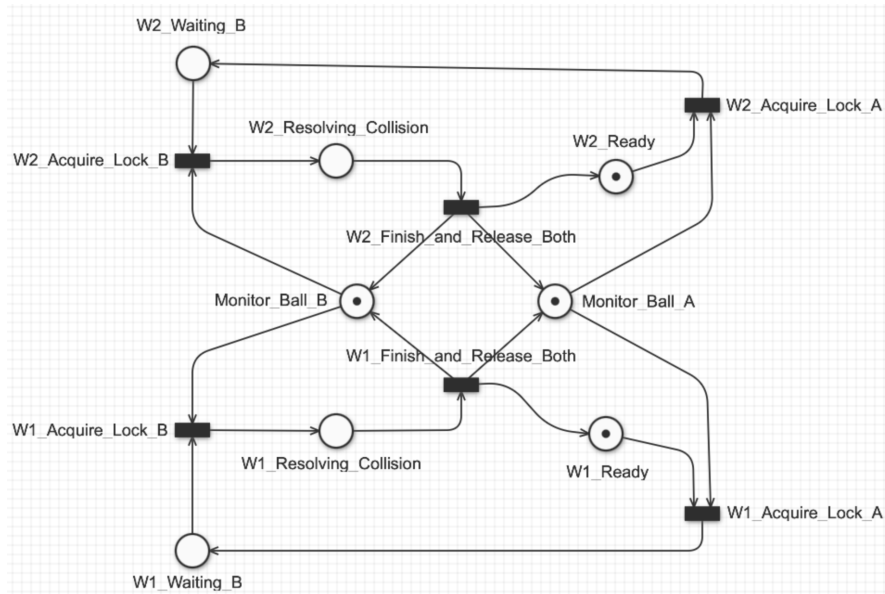


Figure 3: Rete di Petri: Lock Ordering durante la verifica concorrente delle collisioni.

La Figura 3 astrae due worker concorrenti in competizione per le due Ball (`Monitor_Ball_A` e `Monitor_Ball_B`). L'introduzione di un ordine deterministico impone che l'acquisizione del primo lock diventi uno step obbligato e mutuamente esclusivo. Il worker sconfitto nella contesa per il primo lock resta in *Ready*, permettendo al thread vincente di procedere indisturbato verso l'acquisizione dell'ultimo lock. La rete è priva di **Deadlock** poichè non esistono configurazioni raggiungibili della rete in cui subentri l'attesa circolare. L'esecuzione si dimostra ciclica e robusta a prescindere dal livello di asincronia tra i task.

4 Benchmark e Performance

Per validare l'efficacia delle strategie implementate, sono stati eseguiti test su tre scenari (*minimal*, *large*, *massive*) monitorando numero di thread, utilizzo medio CPU e frame-rate medio.

	N. Threads (Peak)	% CPU (Avg)	FPS (Avg)
Sketch01	-----		
- minimal == large:	22	17	120
- massive:	24	16	13
V1 ~ Sequential	-----		
- minimal == large:	24	5	120
- massive:	24	22	30
V1 ~ Multi-threaded	-----		
- minimal == large:	25	7	120
- massive:	31	64	60
V2 ~ Executor	-----		
- minimal == large:	32	8	120
- massive:	32	68	80

* Test da 3 minuti; FPS limitati a 120.

4.1 Analisi dei Risultati

I dati mostrano con chiarezza i limiti delle due versioni soprattutto nello scenario **massive**:

- **V1 Multithreading: barriere cicliche.** Nonostante l'implementazione con barriere cicliche, si raggiunge un frame-rate medio di 50 FPS, con un utilizzo CPU del 64%, evidenziando un miglioramento rispetto alla versione sequenziale ma con margini di ottimizzazione.
- **V2 Executor: migliore scalabilità pratica.** Con partizionamento in chunk e *invokeAll*, la sincronizzazione è ridotta alla barriera finale. Si raggiunge il 68% di CPU media e 80 FPS, migliorando nettamente rispetto all'approccio sequenziale (13 FPS).

4.2 Confronto con Speedup

La metrica di riferimento è: $S_p = \frac{T_s}{T_p}$. Qui T_s è il tempo della baseline sequenziale e T_p il tempo della variante parallela. Non avendo riportato i tempi diretti, a parità di scenario si può usare l'approssimazione tramite frame-rate, valida solo se non ci sono limiti sugli FPS:

$$S_p \approx \frac{\text{FPS}_p}{\text{FPS}_s}$$

Utilizziamo quindi questa relazione nello scenario **massive**:

- V2 rispetto a V1 (Multithreading): $S_{V2/V1} \approx \frac{80}{50} = 1.6$;
- V2 rispetto alla baseline sequenziale **sketch01**: $S_p \approx \frac{80}{13} \approx 6.15$.

Questi risultati evidenziano un miglioramento significativo: V2 ottiene circa $1.6\times$ rispetto a V1 e oltre $6.15\times$ rispetto alla baseline sequenziale, confermando l'efficacia dell'approccio task-based.

5 Verifica del Progetto

Oltre all'osservazione dei risultati di benchmark, è stata condotta una validazione formale del comportamento concorrente mediante **Java Pathfinder (JPF)**.

JPF esplora sistematicamente i possibili interleaving di thread e permette di verificare proprietà di sicurezza e progresso su scenari ridotti ma rappresentativi.

5.1 Casi di Studio Applicati in JPF

I test descritti in `/jpf/README.md` coprono quattro aree principali:

- **TestBoundedBufferSafety:** valida la correttezza funzionale del buffer concorrente (nessuna perdita o duplicazione di elementi) e la consistenza tra conteggio di produzione e consumo.
- **TestBoundedBufferPollNonBlocking:** verifica la semantica di `poll(0)` usata dal Controller, imponendo ritorno immediato su coda vuota e comportamento corretto in presenza di item disponibili.
- **TestCyclicBarrier:** esamina la sincronizzazione della barriera ciclica, accertando che e che nessun thread venga risvegliato prima che tutti i worker abbiano raggiunto il punto di sincronizzazione.
- **TestCollisionDeadlock:** stressa il meccanismo di lock ordering in presenza di collisioni incrociate tra più worker, verificando l'assenza di dipendenze cicliche sui monitor.